SIE / ENTREGA FINAL / ANEXO DEF

Guía de sustentación y preguntas técnicas

Material privado de estudio. No es un anexo obligatorio de entrega.

Control de documento	Detalle
Proyecto	MesaHelp · MVP académico de gestión de solicitudes
Versión del código	0.3.1 · candidata de cierre funcional (11/11/2026)
Tipo de entrega	Trabajo individual según asignación comunicada por docente
Autor / identificador	[COMPLETAR NOMBRE Y CÓDIGO O GRUPO]
Fecha	11 de noviembre de 2026
Estado	Borrador técnico para revisión y validación final en Windows

Este documento describe funciones verificadas por pruebas automatizadas y evidencias previas. Las opciones nuevas de prioridad y ordenamiento requieren validación manual en Opera antes de entregar la versión definitiva.

Contenido

Contenido 2

1. Preparación de la demostración 3

2. Guion oral — aproximadamente 18 a 22 minutos 3

- Apertura (2 minutos) . . . 3
- Actores y flujos (3 minutos) . . . 3
- Demostración operativa (5 minutos) . . . 3
- Explorador, filtros y orden (2 minutos) . . . 3
- Reportes y CSV (2 minutos) . . . 3
- Arquitectura y código (4 minutos) . . . 3
- Pruebas, impacto y límites (2 minutos) . . . 4

3. Archivos que debes saber explicar 4

4. Preguntas probables de Jefferson 5

5. Guion de contingencia y regla de cierre 6

1. Preparación de la demostración

Ventana	Qué mostrar
PowerShell A	Ejecutar Node en nueva carpeta, DATA_DIR aislado, puerto 3013.
PowerShell B	Ejecutar npm test y mostrar 33 pruebas aprobadas.
Opera normal	Coordinador: dashboard, búsqueda/filtros/orden, detalles, cambios de estado, reportes, CSV.
Opera privada	Solicitante: registro, permisos propios y recepción Live.
VS Code	Abrir server.js, db.js, explorer.js, live.js y reports.js; no buscar archivos durante la exposición.
Excel	CSV de ejemplo; importación de Numero como texto si se requieren ceros.

No mostrar credenciales, archivos SQLite, tokens ni códigos de seguimiento ante la audiencia. Preparar una base de datos DEMO separada de la instalación original.

2. Guion oral — aproximadamente 18 a 22 minutos

Apertura (2 minutos)

«Buenos días. Mi proyecto se llama MesaHelp. Es un MVP de mesa de ayuda orientado a organizar y seguir solicitudes. El problema que aborda es la dispersión de incidencias y la dificultad para conocer su estado y sus actuaciones anteriores. El objetivo no es resolver automáticamente las incidencias, sino ofrecer un registro claro, trazable y consultable».

Actores y flujos (3 minutos)

«Distinguimos coordinador, solicitante e invitado. Los usuarios autenticados gestionan sus solicitudes según el rol; el invitado registra sin cuenta y consulta con un código secreto. La autorización la aplica el servidor, no únicamente la interfaz». Abrir dos navegadores y contrastar paneles.

Demostración operativa (5 minutos)

«Voy a crear una solicitud con asunto, categoría, prioridad y descripción. El sistema asigna un número único y estado Abierto. Coordinación puede registrar una observación o cambiar el estado; ambas actuaciones quedan en el historial». Mostrar ticket, nota y Live en otra sesión.

Explorador, filtros y orden (2 minutos)

«El Explorador permite buscar por asunto o número, filtrar por estado, categoría y prioridad y ordenar por actividad o prioridad. Estas acciones no consultan datos de otros usuarios y se mantienen ante actualizaciones Live». Mostrar primero prioridad alta; después ordenar Alta primero y Limpiar filtros.

Reportes y CSV (2 minutos)

«Reportes resume los tickets autorizados por estado, categoría y prioridad. Los indicadores son instantáneas, no tiempos de desempeño. Se puede descargar un CSV utilizable en Excel sin contraseñas ni códigos privados». Mostrar coordinador vs solicitante.

Arquitectura y código (4 minutos)

«Node.js recibe las peticiones en server.js. La persistencia y las consultas autorizadas están en db.js sobre SQLite. Live usa WebSockets como avisos y HTTP para recuperar la vista. El Explorador filtra y ordena nodos HTML autorizados; reports.js genera resúmenes y CSV». Enseñar archivos precisos, no leer cientos de líneas.

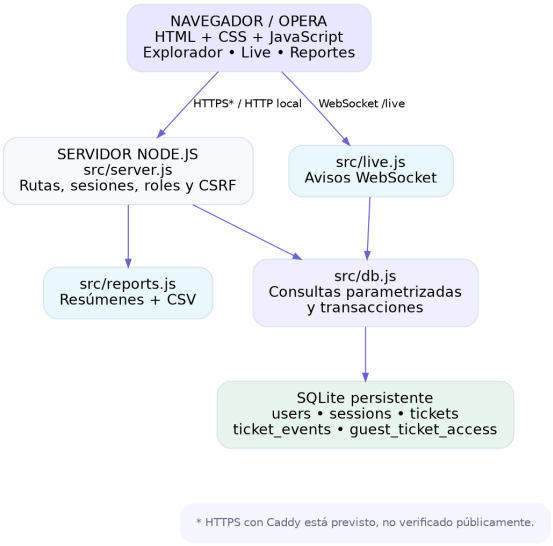


Figura de apoyo. Arquitectura del MVP; no confundir con despliegue público.

Pruebas, impacto y límites (2 minutos)

«La candidata pasó 33 pruebas automatizadas en el entorno de construcción y el sistema anterior ha sido probado manualmente en Opera y Excel. Falta validar visualmente estos dos últimos controles en Windows. El beneficio esperado es mayor trazabilidad y organización; no hemos medido todavía mejoras empresariales. El despliegue público permanece pendiente». Cerrar mostrando la terminal.

3. Archivos que debes saber explicar

Archivo	Explicación precisa en una frase
src/server.js	Controla rutas, sesión, permisos, CSRF y construcción del HTML.
src/db.js	Define tablas e integridad; inserciones transaccionales y consultas SQL parametrizadas.
src/live.js	Notifica mediante WebSocket solo a sesiones que siguen autorizadas.
public/live.js	Recibe avisos y actualiza datos vía HTTP; protege campos no guardados.
public/explorer.js	Filtra y ordena elementos del listado ya autorizado por servidor.
src/reports.js	Agrupa tickets autorizados y escapa CSV ante fórmulas de Excel.
scripts/backup-db.cjs	VACUUM INTO genera una copia de SQLite consistente.
Dockerfile	Empaqueta servidor, lo ejecuta como usuario no root y ofrece healthcheck.

4. Preguntas probables de Jefferson

Pregunta	Respuesta que debes manejar
¿Qué problema resuelve?	Dispersión de solicitudes y falta de trazabilidad; no arregla incidencias automáticamente.
¿Por qué es sistema de información?	Captura, organiza, transforma y ofrece información para un proceso de gestión.
¿Por qué Node.js?	Permite servidor web y lógica de rutas con las bibliotecas nativas usadas en este MVP.
¿Por qué SQLite?	Sencillez local, persistencia transaccional y ausencia de servidor de BD adicional; límites para alta concurrencia.
¿Qué tablas hay?	users, sessions, tickets, ticket_events, guest_ticket_access.
¿Cómo se protege un ticket ajeno?	Consulta por requester_id y verificación en rutas; coordinador consulta todo.
¿Qué hace CSRF?	Impide acciones POST no autorizadas usando tokens de formulario y sesión.
¿Por qué no guardar contraseñas claras?	Se utiliza scrypt con sal; el hash protege frente a exposición directa.
¿Cómo funciona Live?	WebSocket avisa y cliente hace GET HTTP autorizado; no envía todo el contenido por el socket.
¿Qué pasa si Live falla?	Existen reconexión y sondeo; el usuario también puede recargar.
¿Qué guarda el historial?	Actor, estado previo/nuevo, nota y momento.
¿Qué datos exporta CSV?	ID, asunto, solicitante, categoría, prioridad, estado y fechas; no códigos privados ni descripción.
¿CSV respeta filtros visuales?	No. Exporta todos los registros que el servidor autoriza al usuario.
¿Por qué los reportes no son KPIs de SLA?	No se definieron ni probaron tiempos de respuesta/resolución como métricas de desempeño.
¿Qué cambió en la última versión?	Filtro por prioridad y ordenamiento del Explorador; 4 pruebas nuevas; no migración SQLite.
¿Cuántas pruebas tienen?	33/33 en la candidata, además de las verificaciones manuales previas. No equivalen a certificación.
¿Qué probó Docker?	En una versión anterior, ejecución local y persistencia con volumen; el nuevo build queda por validar.
¿Ya funciona en Hetzner?	No. Hay configuración propuesta con Caddy pero no despliegue externo validado.
¿Cómo recuperan una contraseña?	No existe autoservicio; debe gestionarlo operador; límite de MVP.
¿Qué falta para producción?	Revisión seguridad, WebSocket mantenido, operación, monitoreo, backups, TLS, credenciales, pruebas de carga y privacidad.

5. Guion de contingencia y regla de cierre

- Si un puerto está ocupado: localizar proceso, no matar procesos desconocidos; elegir otro puerto libre y usar el mismo en Opera.
- Si una sesión expira: volver a /login y autenticarse sin exponer la contraseña.
- Si Live tarda: confirmar servidor y conexión, esperar la actualización o refrescar manualmente; no simular un resultado.
- Si Excel interpreta `0006` como 6: importar Numero como texto y explicar el tipo de dato.
- Si preguntan por GitHub o despliegue público: dar la situación real, nunca inventar URL o funciones.

Frase final: «MesaHelp demuestra un flujo trazable de registro, atención y análisis de solicitudes. Se encuentra validado como prototipo académico local, con límites claros y un plan de evolución documentado».